

Database System Project2.

Normalization and Query Processing

전공: 수학

20221197 백서연

함수 종속성과 BCNF 설명

BCNF (Boyce-Codd Normal Form)는 데이터베이스 정규화의 높은 단계입니다. BCNF를 만족하려면, 테이블의 모든 결정자가 후보 키여야 합니다. BCNF 만족 여부를 확인하려면 각 테이블의 함수 종속성을 파악하고 모든 함수 종속성의 결정자가 후보 키인지 확인해야 합니다.

Address 테이블

함수 종속성:

- `address\_id -> street, city, district`

설명:

`address\_id`는 기본 키이자 후보 키입니다. `address\_id`는 모든 다른 속성(street, city, district)을 결정합니다. 따라서 `Address` 테이블은 BCNF를 만족합니다.

Buyer 테이블

함수 종속성:

- `buyer\_id -> buyer\_name, buyer\_contact`

설명:

`buyer\_id`는 기본 키이자 후보 키입니다. `buyer\_id`는 모든 다른 속성(buyer_name, buyer_contact)을 결정합니다. 따라서 `Buyer` 테이블은 BCNF를 만족합니다.

Seller 테이블

함수 종속성:

- `seller\_id` -> seller_name, seller_contact`

설명:

`seller\_id`는 기본 키이자 후보 키입니다. `seller\_id`는 모든 다른 속성(seller_name, seller_contact)을 결정합니다. 따라서 `Seller` 테이블은 BCNF를 만족합니다.

Property 테이블

함수 종속성:

- `property\_id` -> address_id, seller_id, property_price, property_bathroom, property_bedroom, property_area, school_district, list_date, type, status`

- `address\_id` -> city, district`

- `seller\_id` -> seller_name, seller_contact`

설명:

1. 기본 키 `property\_id`는 `Property` 테이블의 모든 다른 속성을 결정합니다. 그러나 추가적인 종속성도 있습니다. 예를 들어 `address\_id`는 city와 district를 결정하고, `seller\_id`는 seller_name과 seller_contact를 결정합니다.

2. 후보 키인 `property\_id`가 모든 속성을 결정하므로 `Property` 테이블은 BCNF를 만족

합니다.

Sale 테이블

함수 종속성:

- `sale\_id` -> `property\_id`, `buyer\_id`, `seller\_id`, `address\_id`, `agent\_id`, `sale\_price`, `sale\_date`
- `property\_id` -> `address\_id`, `seller\_id`
- `buyer\_id` -> `buyer\_name`, `buyer\_contact`
- `seller\_id` -> `seller\_name`, `seller\_contact`
- `agent\_id` -> `agent\_name`, `agent\_contact`

설명:

1. 기본 키 `sale\_id`는 `Sale` 테이블의 모든 다른 속성을 결정합니다.
2. 추가적인 종속성으로 `property\_id`는 `address\_id`와 `seller\_id`를 결정하고, `buyer\_id`, `seller\_id`, `agent\_id`는 각각 그들의 이름과 연락처를 결정합니다.
3. `sale\_id`가 모든 속성을 결정하므로 `Sale` 테이블은 BCNF를 만족합니다.

Agent 테이블

함수 종속성:

- `agent\_id` -> `agent\_name`, `agent\_contact`

설명:

`agent\_id`는 기본 키이자 후보 키입니다. `agent\_id`는 모든 다른 속성(`agent\_name`, `agent\_contact`)을 결정합니다. 따라서 `Agent` 테이블은 BCNF를 만족합니다.

Photo 테이블

함수 종속성:

- `photo\_id` -> `property\_id`, `type`, `image`, `description`
- `property\_id` -> `address\_id`

설명:

1. 기본 키 `photo\_id`는 `Photo` 테이블의 모든 다른 속성을 결정합니다.
2. 추가적인 종속성으로 `property\_id`는 `address\_id`를 결정합니다.
3. `photo\_id`가 모든 속성을 결정하므로 `Photo` 테이블은 BCNF를 만족합니다.

BCNF 검증 절차

1. ****함수 종속성 식별****: 각 테이블의 모든 함수 종속성을 나열합니다.
2. ****후보 키 결정****: 각 테이블의 모든 후보 키를 식별합니다.
3. ****BCNF 준수 여부 확인****: 모든 함수 종속성 $X \rightarrow Y$ 에 대해 X 가 후보 키인지 확인합니다.

결론

주어진 스키마의 모든 테이블(`Address`, `Buyer`, `Seller`, `Property`, `Sale`, `Agent`, `Photo`)이 BCNF를 만족합니다. 각 테이블의 기본 키는 테이블의 모든 다른 속성을 결정하며, 비자명한 함수 종속성이 BCNF 규칙을 위반하지 않습니다.

이와 같은 상세한 설명과 검증 절차는 데이터베이스 스키마가 잘 설계되어 있으며 BCNF를 준수하여 데이터 무결성과 최소한의 중복성을 보장함을 확인합니다.

ODBC Implementation within MySQL

ODBC 환경 설정 및 초기화

ODBC(Open Database Connectivity) 환경과 MySQL 데이터베이스 연결을 설정하는 절차는 몇 가지 주요 단계로 구성됩니다. 이 과정은 데이터베이스와의 연결을 설정하고, 쿼리를 실행하며, 결과를 처리하는 데 필수적입니다.

먼저, 프로그램에서 ODBC 환경을 설정하고 데이터베이스에 연결하기 위해 필요한 라이브러리와 헤더 파일을 포함합니다. 여기서 `#define \_CRT\_SECURE\_NO\_WARNINGS`는 보안 경고를 무시하기 위해 사용되고, `mysql.h`는 MySQL 관련 함수들을 사용하기 위해 포함합니다.

프로그램의 초기 단계에서는 MySQL 서버의 호스트, 사용자 이름, 비밀번호, 그리고 데이터베이스 이름을 지정합니다. 이는 이후 데이터베이스에 연결할 때 사용됩니다.

```
```\n\nconst char* host = "127.0.0.1";\n\nconst char* user = "root";\n\nconst char* pw = "asdf";\n\nconst char* db = "project";\n\n```\n
```

MySQL 연결을 초기화하기 위해 `mysql\_init` 함수를 사용하여 MYSQL 객체를 초기화합니다. 그런 다음 `mysql\_real\_connect` 함수를 사용하여 MySQL 서버에 연결합니다. 이 함수는 서버 호스트, 사용자 이름, 비밀번호, 데이터베이스 이름, 그리고 포트 번호를 인수로 받아 데이터베이스 연결을 설정합니다. 연결이 성공하면 "Connection Succeed" 메시지를 출력하고, 실패하면 오류 메시지를 출력합니다.

## #### Query 실행 및 결과 처리

데이터베이스에 성공적으로 연결되면 사용자가 다양한 쿼리를 실행할 수 있도록 메뉴를 제공합니다. 각 쿼리 유형에 따라 적절한 SQL 쿼리를 작성하고 실행합니다. 쿼리를 실행하고 결과를 처리하는 과정은 다음과 같습니다:

### 1. \*\*쿼리 실행\*\*:

`mysql\_query` 함수를 사용하여 SQL 쿼리를 실행합니다. 이 함수는 실행할 SQL 쿼리를 인수로 받아 데이터베이스에서 해당 쿼리를 실행합니다.

### 2. \*\*결과 집합 가져오기\*\*:

`mysql\_store\_result` 함수를 사용하여 쿼리 실행 결과를 가져옵니다. 이 함수는 쿼리 결과 집합을 MYSQL\_RES 구조체로 반환합니다. 만약 결과 집합을 가져오지 못하면 오류 메시지를 출력합니다.

### 3. \*\*결과 집합 처리\*\*:

결과 집합을 처리하기 위해 `mysql\_num\_fields` 함수를 사용하여 결과 집합의 필드(컬럼) 수를 가져옵니다. 그런 다음 `mysql\_fetch\_fields` 함수를 사용하여 필드 정보를 가져와 출력합니다.

각 행의 데이터를 처리하기 위해 `mysql\_fetch\_row` 함수를 사용하여 결과 집합의 각 행을 순차적으로 가져옵니다. 각 행의 데이터를 `mysql\_fetch\_row` 함수의 반환 값인 MYSQL\_ROW 배열을 통해 접근합니다. 각 컬럼의 데이터는 문자열 형태로 출력됩니다.

### 4. \*\*결과 해제\*\*:

결과 집합 처리가 완료되면 `mysql\_free\_result` 함수를 사용하여 결과 집합에 할당된 메모리를 해제합니다.

이와 같은 과정을 통해 다양한 쿼리를 실행하고 그 결과를 출력할 수 있습니다. 예를 들어, 'Mapo' 구역의 판매 중인 주택의 주소를 찾는 쿼리는 다음과 같습니다:

```
```c
```

```
sprintf(query, "SELECT a.street, a.city, a.district FROM Property p JOIN Address a ON  
p.address_id = a.address_id WHERE a.district = 'Mapo';");
```

```
execute_query(connection, query);
```

```
```
```

사용자는 각 쿼리 유형에 따라 적절한 SQL 쿼리를 작성하고 `execute\_query` 함수를 호출하여 쿼리를 실행합니다. 이 함수는 쿼리를 실행하고 결과를 출력하는 모든 과정을 수행합니다.

프로그램은 사용자가 다양한 쿼리 유형을 선택할 수 있도록 메뉴를 제공하며, 각 쿼리 유형에 대한 서브 쿼리도 제공합니다. 예를 들어, 'Mapo' 구역의 주택 중 가격이 10억 원에서 15억 원 사이인 주택을 찾는 서브 쿼리가 있습니다.

마지막으로, 프로그램이 종료될 때 `mysql\_close` 함수를 사용하여 MySQL 연결을 닫고, 모든 리소스를 해제합니다. 이를 통해 데이터베이스와의 연결을 안전하게 종료할 수 있습니다.

이와 같은 ODBC 환경 설정 및 초기화 과정, 쿼리 실행 및 결과 처리 과정을 통해 ODBC를 사용하여 MySQL 데이터베이스에 연결하고 데이터를 조회하는 방법을 이해할 수 있습니다. 각 단계에서 발생할 수 있는 오류를 처리하기 위한 기본적인 오류 처리 로직도 포함되어 있으며, 필요에 따라 추가적인 쿼리 실행 및 결과 처리 예제를 포함시켜 실제 응용 프로그램에서의 활용 방법을 더욱 명확히 할 수 있습니다.

## CPP코드 분석

### ### 데이터베이스와 테이블 생성

먼저, MySQL 데이터베이스에 연결하고 필요한 테이블을 생성했습니다. 이를 위해 `mysql\_query` 함수를 사용하여 SQL 명령어를 실행했습니다. `Address`, `Buyer`, `Seller`, `Property`, `Sale`, `Agent`, `Photo` 테이블을 생성하고 외래 키 제약 조건을 설정했습니다. 각 테이블에 대해 적절한 데이터 타입과 제약 조건을 정의하여 데이터 무결성을 유지했습니다.

### ### 데이터 삽입

각 테이블에 데이터를 삽입했습니다. 예를 들어, `Address` 테이블에는 주소 데이터를 삽입하고, `Buyer`와 `Seller` 테이블에는 구매자와 판매자 정보를 삽입했습니다. `Property` 테이블에는 부동산 정보를, `Sale` 테이블에는 거래 정보를 삽입했습니다. `Agent` 테이블에는 에이전트 정보를, `Photo` 테이블에는 부동산 사진 정보를 삽입했습니다. 삽입된 데이터는 이후 쿼리에서 사용했습니다.

### ### 쿼리 실행

프로그램은 사용자가 선택할 수 있는 여러 종류의 쿼리를 제공했습니다. 사용자는 콘솔에서 원하는 쿼리 타입을 선택하고, 필요한 추가 정보를 입력했습니다.

### #### TYPE 1: 특정 지역의 판매 중인 집 주소 찾기

먼저, 'Mapo' 지역의 판매 중인 집 주소를 찾는 쿼리를 실행했습니다. `Property`와 `Address` 테이블을 조인하여 해당 지역의 주소를 검색했습니다. 결과는 화면에 출력했습니다. 이후 사용자가 서브 타입을 선택하면 가격 범위 내의 집을 찾는 추가 쿼리를 실행했습니다.

#### ##### TYPE 2: 특정 학군 내의 집 주소 찾기

8학군 내의 집 주소를 찾는 쿼리를 실행했습니다. `Property`와 `Address` 테이블을 조인하여 해당 학군 내의 집 주소를 검색했습니다. 결과는 화면에 출력했습니다. 이후 사용자가 서브 타입을 선택하면 특정 조건(예: 침실 4개 이상, 욕실 2개 이상)에 맞는 집을 찾는 추가 쿼리를 실행했습니다.

#### ##### TYPE 3: 특정 연도에 가장 많은 부동산을 판매한 에이전트 찾기

2022년에 가장 많은 부동산을 판매한 에이전트를 찾는 쿼리를 실행했습니다. `Agent`와 `Sale` 테이블을 조인하여 해당 연도의 판매 총액을 계산하고, 이를 기준으로 정렬하여 가장 많은 판매를 한 에이전트를 찾았습니다. 결과는 화면에 출력했습니다. 사용자는 서브 타입을 선택하여 2023년의 상위 k명 에이전트나 2021년의 하위 10% 에이전트를 찾는 추가 쿼리를 실행했습니다.

#### ##### TYPE 4: 2022년에 판매된 부동산의 평균 판매 가격 및 평균 시장 기간 계산

에이전트별로 2022년에 판매된 부동산의 평균 판매 가격과 평균 시장 기간을 계산하는 쿼리를 실행했습니다. `Agent`, `Sale`, `Property` 테이블을 조인하여 평균값을 계산하고, 이를 화면에 출력했습니다. 사용자는 서브 타입을 선택하여 2023년에 판매된 부동산의 최대 판매 가격이나 각 에이전트별로 시장에 가장 오래 있었던 부동산을 찾는 추가 쿼리를 실행했습니다.

#### ##### TYPE 5: 가장 비싼 부동산 사진 보여주기

가장 비싼 스튜디오, 원룸, 다세대 주택, 단독 주택의 사진을 보여주는 쿼리를 실행했습니다. `Property`와 `Photo` 테이블을 조인하여 각 유형별로 가장 비싼 부동산의 사진을 검색했습니다. 결과는 화면에 출력했습니다. 사용자가 쿼리를 반복해서 실행할 수 있도록

옵션을 제공했습니다.

#### #### TYPE 6: 부동산 판매 기록 추가

특정 부동산의 판매를 기록하는 쿼리를 실행했습니다. 사용자는 부동산 ID, 판매 가격, 구매자 ID, 판매 에이전트 ID, 구매 에이전트 ID, 판매 날짜를 입력했습니다. 입력된 정보를 기반으로 `Property` 테이블의 상태를 'sold'로 업데이트하고, `Sale` 테이블에 새로운 판매 기록을 추가했습니다.

#### #### TYPE 7: 새로운 에이전트 추가

새로운 에이전트를 데이터베이스에 추가하는 쿼리를 실행했습니다. 사용자는 에이전트 이름과 연락처를 입력했습니다. 입력된 정보를 기반으로 `Agent` 테이블에 새로운 에이전트 기록을 추가했습니다.

---

이와 같은 과정을 통해 MySQL 데이터베이스와 ODBC를 사용하여 다양한 쿼리를 실행하고 결과를 사용자에게 보여줄 수 있었습니다.

## 쿼리 분석

우선 데이터베이스를 생성하고 사용했습니다. 데이터베이스 이름은 `project`이며, 기존에 동일한 이름의 데이터베이스가 존재할 경우 삭제하고 새로 생성했습니다. 이 후, `project` 데이터베이스를 사용하도록 설정했습니다.

#### ### Address 테이블

`Address` 테이블은 주소 정보를 저장했습니다. 이 테이블은 `address\_id`, `street`, `city`, `district` 필드를 포함하며, `address\_id`가 기본 키입니다. 각 주소는 고유한 `address\_id`를 가지며, 거리(`street`), 도시(`city`), 구역(`district`) 정보를 저장했습니다.

#### ### Buyer 테이블

`Buyer` 테이블은 구매자 정보를 저장했습니다. 이 테이블은 `buyer\_id`, `buyer\_name`, `buyer\_contact` 필드를 포함하며, `buyer\_id`가 기본 키입니다. 각 구매자는 고유한 `buyer\_id`를 가지며, 이름(`buyer\_name`)과 연락처(`buyer\_contact`) 정보를 저장했습니다.

#### ### Seller 테이블

`Seller` 테이블은 판매자 정보를 저장했습니다. 이 테이블은 `seller\_id`, `seller\_name`, `seller\_contact` 필드를 포함하며, `seller\_id`가 기본 키입니다. 각 판매자는 고유한 `seller\_id`를 가지며, 이름(`seller\_name`)과 연락처(`seller\_contact`) 정보를 저장했습니다.

#### ### Property 테이블

`Property` 테이블은 부동산 정보를 저장했습니다. 이 테이블은 `property\_id`, `address\_id`, `seller\_id`, `property\_price`, `property\_bathroom`, `property\_bedroom`, `property\_area`, `school\_district` 필드를 포함하며, `property\_id`가 기본 키입니다. 또한 `address\_id`와 `seller\_id`는 각각 `Address`와 `Seller` 테이블의 외래 키로 설정했습니다. 이 테이블은 부동산의 가격(`property\_price`), 욕실 수(`property\_bathroom`), 침실 수(`property\_bedroom`), 면적(`property\_area`), 학군(`school\_district`) 정보를 저장했습니다.

#### ### Sale 테이블

`Sale` 테이블은 부동산 판매 정보를 저장했습니다. 이 테이블은 `sale\_id`, `property\_id`, `buyer\_id`, `seller\_id`, `address\_id`, `agent\_id`, `sale\_price`, `sale\_date` 필드를 포함하며,

`sale\_id`가 기본 키입니다. `property\_id`, `buyer\_id`, `seller\_id`, `address\_id`는 각각 `Property`, `Buyer`, `Seller`, `Address` 테이블의 외래 키로 설정했습니다. 이 테이블은 판매 가격(`sale\_price`)과 판매 날짜(`sale\_date`) 정보를 저장했습니다.

### ### Agent 테이블

`Agent` 테이블은 에이전트 정보를 저장했습니다. 이 테이블은 `agent\_id`, `agent\_name`, `agent\_contact` 필드를 포함하며, `agent\_id`가 기본 키입니다. 각 에이전트는 고유한 `agent\_id`를 가지며, 이름(`agent\_name`)과 연락처(`agent\_contact`) 정보를 저장했습니다.

### ### Photo 테이블

`Photo` 테이블은 부동산 사진 정보를 저장했습니다. 이 테이블은 `photo\_id`, `property\_id`, `type`, `image`, `description` 필드를 포함하며, `photo\_id`가 기본 키입니다. `property\_id`는 `Property` 테이블의 외래 키로 설정했습니다. 이 테이블은 사진의 유형(`type`), 이미지 데이터(`image`), 설명(`description`) 정보를 저장했습니다.

### ### 데이터 삽입

각 테이블에 데이터를 삽입했습니다. `Address` 테이블에는 여러 주소를 삽입하고, `Buyer`와 `Seller` 테이블에는 각각 여러 명의 구매자와 판매자 정보를 삽입했습니다. `Property` 테이블에는 다양한 부동산 정보를 삽입했습니다. 예를 들어, 특정 주소와 판매자, 가격, 욕실 수, 침실 수, 면적, 학군 정보를 포함한 부동산 정보를 삽입했습니다. `Sale` 테이블에는 부동산 거래 정보를 삽입했습니다. `Agent` 테이블에는 여러 에이전트 정보를 삽입하고, `Photo` 테이블에는 부동산 사진 정보를 삽입했습니다.

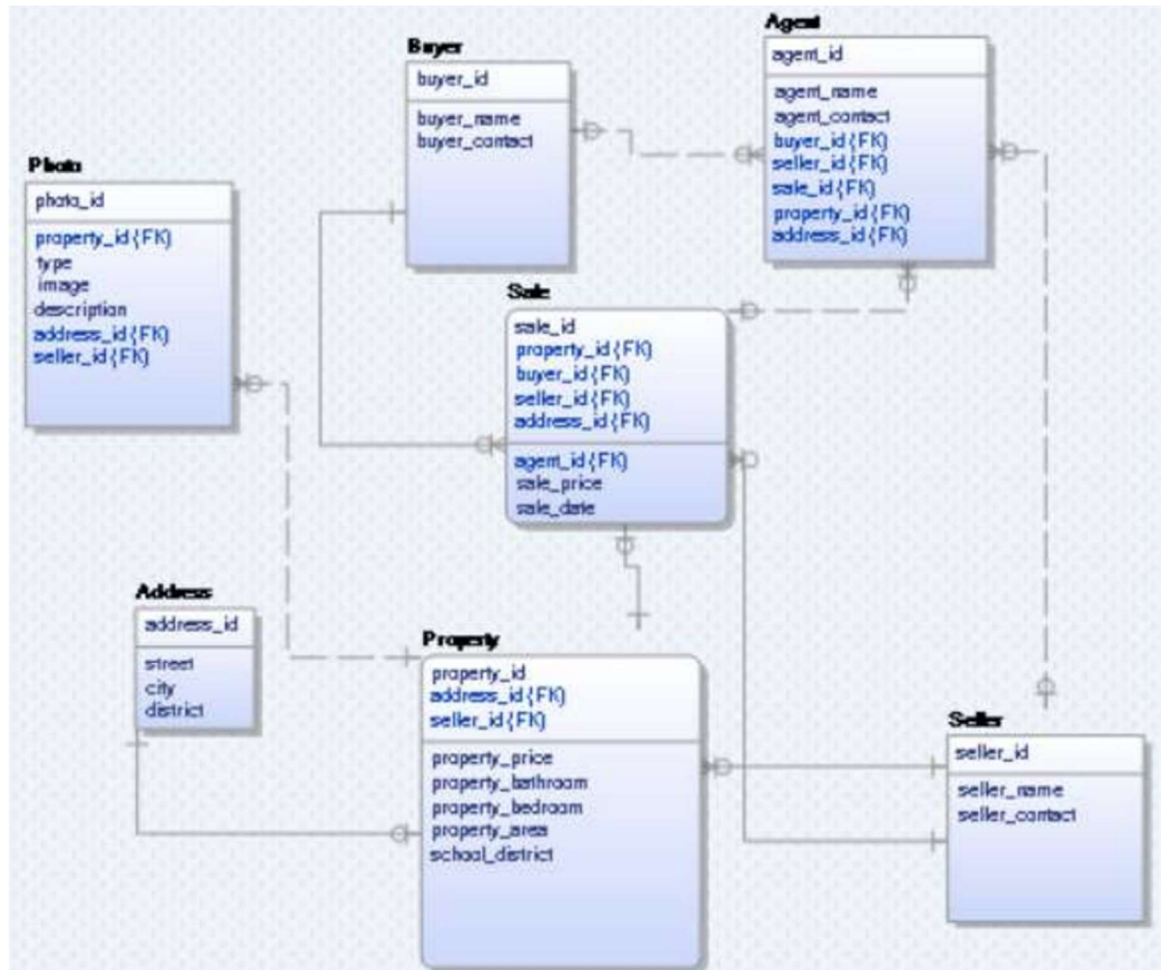
### ### 추가 컬럼 및 데이터 업데이트

`Property` 테이블에 `list\_date`라는 새로운 컬럼을 추가하고, 각 부동산의 리스트 날짜를

업데이트했습니다. 또한 `Property` 테이블에 `type`이라는 새로운 컬럼을 추가하여 부동산의 유형(예: 스튜디오, 원룸, 다세대 주택, 단독 주택)을 설정했습니다. `Property` 테이블에 `status`라는 새로운 컬럼을 추가하여 부동산의 상태를 나타내도록 했으며, 기본값은 'available'로 설정했습니다.

`Sale` 테이블과 `Agent` 테이블의 `sale\_id`와 `agent\_id` 필드는 자동 증가 (AUTO\_INCREMENT)로 설정했습니다. 이를 통해 새로운 판매 기록이나 에이전트 기록이 추가될 때 자동으로 ID가 증가하도록 했습니다.

이와 같은 쿼리들을 통해 데이터베이스와 테이블을 생성하고, 데이터를 삽입하며, 필요한 컬럼을 추가 및 업데이트하여 데이터베이스를 완성했습니다. 이러한 과정을 통해 부동산 데이터베이스를 효율적으로 관리할 수 있습니다.



### ### 논리적 스키마 분석

#### #### 테이블 설명 및 분석

##### \*\*1. Address 테이블\*\*

- **\*\*컬럼\*\***: address\_id (PK), street, city, district
- **\*\*설명\*\***: 주소 정보를 저장합니다. 'address\_id'는 각 주소의 고유 식별자로 기본 키입니다. 'street', 'city', 'district'는 주소의 세부 정보를 나타냅니다.
- **\*\*설계 이유\*\***: 주소 정보를 별도로 분리함으로써 데이터 중복을 최소화하고, 주소를 효율적으로 관리할 수 있습니다.

## **\*\*2. Buyer 테이블\*\***

- **\*\*컬럼\*\***: buyer\_id (PK), buyer\_name, buyer\_contact
- **\*\*설명\*\***: 구매자 정보를 저장합니다. `buyer\_id`는 각 구매자의 고유 식별자로 기본 키입니다. `buyer\_name`과 `buyer\_contact`는 각각 구매자의 이름과 연락처를 나타냅니다.
- **\*\*설계 이유\*\***: 구매자 정보를 별도로 관리하여 구매자에 대한 데이터를 체계적으로 관리할 수 있습니다.

## **\*\*3. Seller 테이블\*\***

- **\*\*컬럼\*\***: seller\_id (PK), seller\_name, seller\_contact
- **\*\*설명\*\***: 판매자 정보를 저장합니다. `seller\_id`는 각 판매자의 고유 식별자로 기본 키입니다. `seller\_name`과 `seller\_contact`는 각각 판매자의 이름과 연락처를 나타냅니다.
- **\*\*설계 이유\*\***: 판매자 정보를 별도로 관리하여 판매자에 대한 데이터를 체계적으로 관리할 수 있습니다.

## **\*\*4. Property 테이블\*\***

- **\*\*컬럼\*\***: property\_id (PK), address\_id (FK), seller\_id (FK), property\_price, property\_bathroom, property\_bedroom, property\_area, school\_district
- **\*\*설명\*\***: 부동산 정보를 저장합니다. `property\_id`는 각 부동산의 고유 식별자로 기본 키입니다. `address\_id`는 `Address` 테이블을 참조하는 외래 키이며, `seller\_id`는 `Seller` 테이블을 참조하는 외래 키입니다. 부동산의 가격, 욕실 수, 침실 수, 면적, 학군 정보를 포함합니다.
- **\*\*설계 이유\*\***: 부동산과 관련된 다양한 정보를 체계적으로 관리하고, 다른 테이블과의 관계를 통해 데이터를 효율적으로 참조할 수 있도록 합니다.

#### **\*\*5. Sale 테이블\*\***

- **\*\*컬럼\*\***: sale\_id (PK), property\_id (FK), buyer\_id (FK), seller\_id (FK), address\_id (FK), agent\_id (FK), sale\_price, sale\_date
- **\*\*설명\*\***: 부동산 거래 정보를 저장합니다. `sale\_id`는 각 거래의 고유 식별자로 기본 키입니다. `property\_id`, `buyer\_id`, `seller\_id`, `address\_id`, `agent\_id`는 각각 `Property`, `Buyer`, `Seller`, `Address`, `Agent` 테이블을 참조하는 외래 키입니다. 거래 가격과 거래 날짜를 포함합니다.
- **\*\*설계 이유\*\***: 부동산 거래 내역을 체계적으로 관리하고, 관련된 여러 테이블과의 관계를 통해 데이터를 효율적으로 조회할 수 있도록 합니다.

#### **\*\*6. Agent 테이블\*\***

- **\*\*컬럼\*\***: agent\_id (PK), agent\_name, agent\_contact
- **\*\*설명\*\***: 부동산 에이전트 정보를 저장합니다. `agent\_id`는 각 에이전트의 고유 식별자로 기본 키입니다. `agent\_name`과 `agent\_contact`는 각각 에이전트의 이름과 연락처를 나타냅니다.
- **\*\*설계 이유\*\***: 에이전트 정보를 별도로 관리하여 에이전트에 대한 데이터를 체계적으로 관리할 수 있습니다.

#### **\*\*7. Photo 테이블\*\***

- **\*\*컬럼\*\***: photo\_id (PK), property\_id (FK), type, image, description
- **\*\*설명\*\***: 부동산 사진 정보를 저장합니다. `photo\_id`는 각 사진의 고유 식별자로 기본 키입니다. `property\_id`는 `Property` 테이블을 참조하는 외래 키입니다. 사진의 유형, 이미지 데이터, 설명을 포함합니다.
- **\*\*설계 이유\*\***: 부동산과 관련된 사진 정보를 체계적으로 관리하고, 부동산 정보와 연계하여 사진을 효율적으로 조회할 수 있도록 합니다.

#### #### 논리적 스키마 설계 이유

이 논리적 스키마는 데이터 중복을 최소화하고, 데이터 무결성을 유지하며, 각 테이블 간의 관계를 명확하게 정의하여 데이터베이스의 성능과 유지보수성을 높이기 위해 설계되었습니다. 각각의 엔터티(구매자, 판매자, 부동산, 거래, 에이전트, 사진, 주소)를 독립된 테이블로 관리함으로써 데이터의 일관성을 유지하고, 효율적인 데이터 관리를 할 수 있습니다.

특히, 외래 키를 통해 각 테이블 간의 관계를 명확히 정의하여 데이터를 쉽게 참조하고 조작할 수 있도록 하였습니다. 예를 들어, `Property` 테이블은 `Address`와 `Seller` 테이블을 참조하여 부동산의 위치와 판매자 정보를 관리하고, `Sale` 테이블은 `Property`, `Buyer`, `Seller`, `Address`, `Agent` 테이블을 참조하여 거래 내역을 관리합니다.

이와 같은 설계는 데이터의 무결성을 유지하면서도 필요한 정보를 효율적으로 조회할 수 있도록 하여, 부동산 관리 시스템의 기능을 효과적으로 지원합니다.